

Portfolio

프로젝트 순서

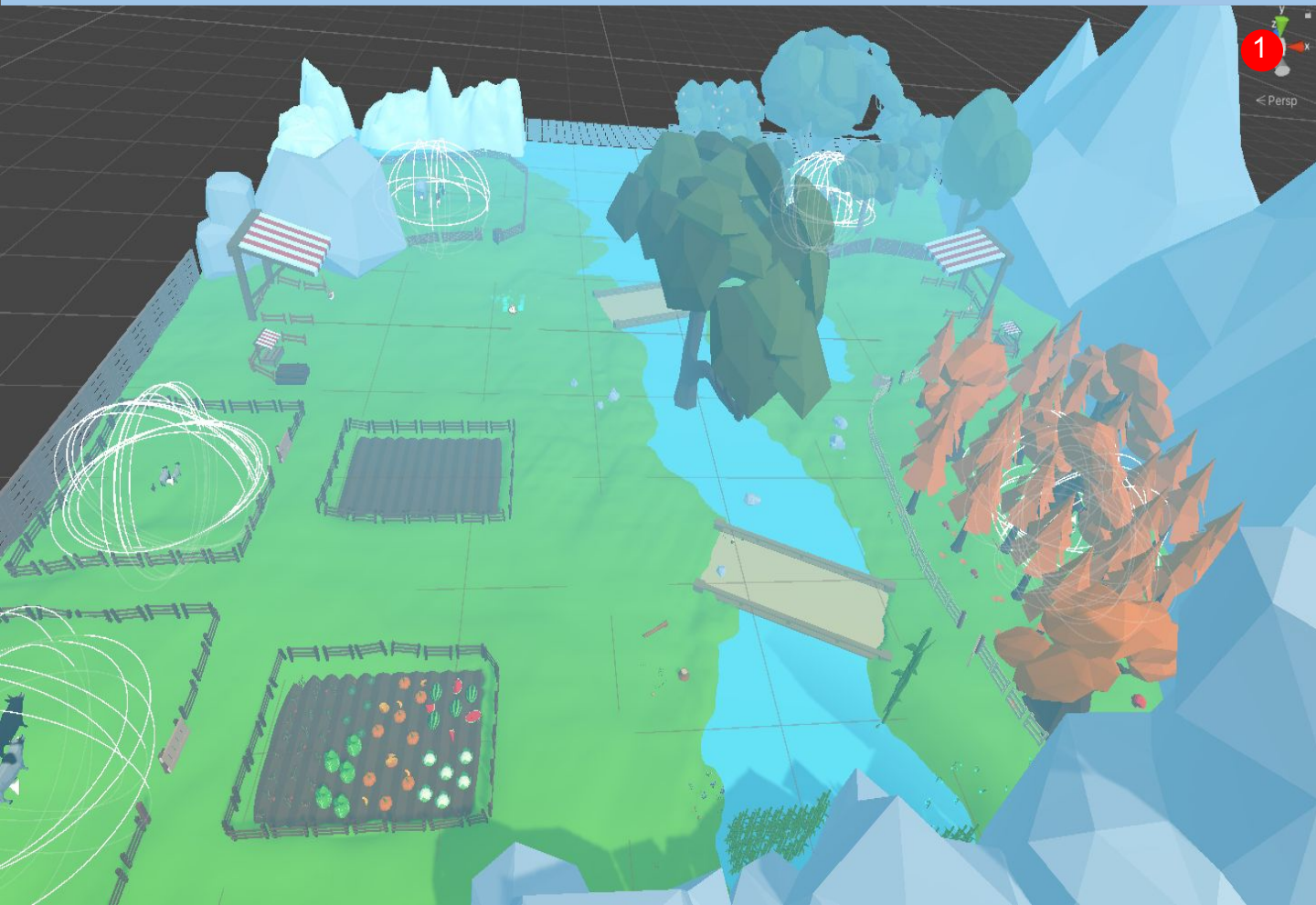
Zoo

ISSUE

Zoo

- 프로젝트 제작기간 : 22.06 ~ 22.07
- 팀원 : 강창민 / 김민주
- 담당 업무 : 월드맵 제작, 상태창 UI 및 상호작용 구현, 플레이어와 오브젝트 상호작용
- 사용 언어 : C#
- 툴 : Unity 3D, Visual studio code

구성 요소



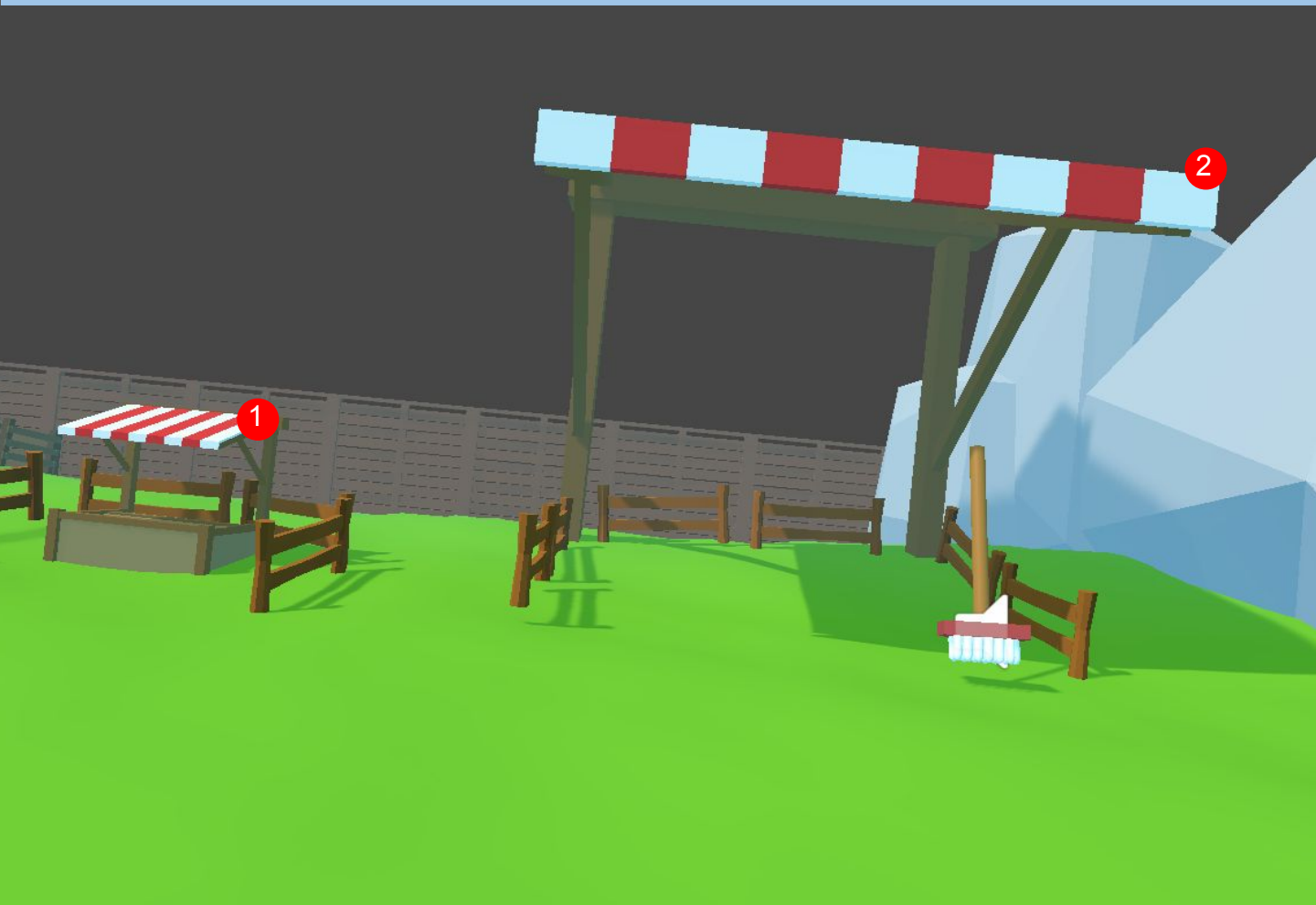
기능 설명

[1] - 맵 디자인

전체 맵 디자인입니다.

동물 농장을 기반으로 디자인 했고 오브젝트는 ASSET STORE에서 가져와서 사용하였습니다.

구성 요소



기능 설명

[1] - 먹이 생성 시스템

- 특정 구역을 빈 오브젝트로 설정하고 오브젝트의 자식으로 과일(먹이)을 위치하여 게임 플레이시 과일(먹이)를 생성하도록 구현

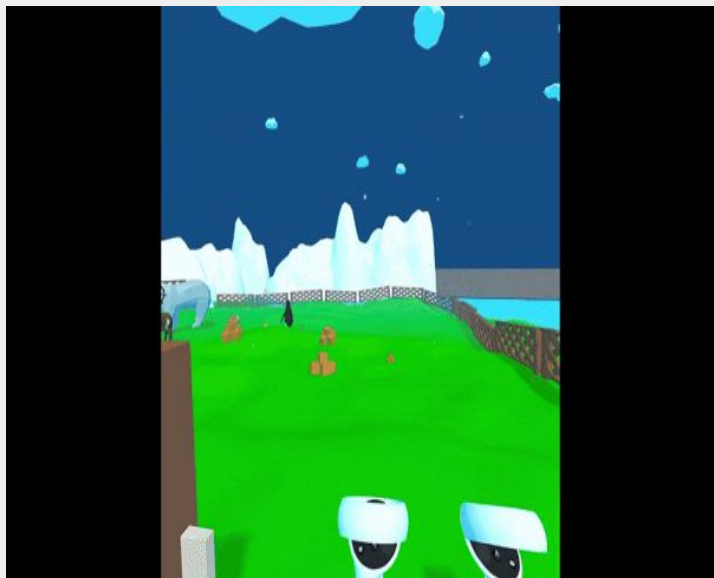
- 과일(먹이)는 설정한 시간에 따라 재생성

[2] - 동물 목욕 구역

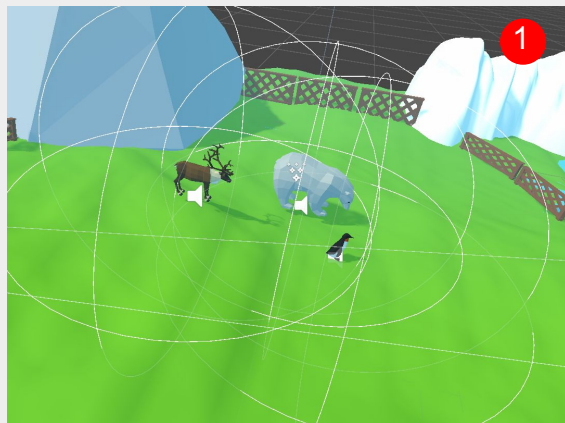
- 청소 구역에 구역 설정을 하여 동물을 데려올 경우 움직이지 못하고 가만히 있게 설정했습니다.

- 해당 구역에 들어오면 Oculus 기기를 통해 동물을 깨끗하게 할 수 있습니다.

시연



구성 요소



기능 설명

[1] - 동물 인식 범위

- Search Range를 정의하여 동물 마다 개별적 인식 범위 설정 가능

- 범위 안에 들어온 오브젝트 중 Player 태그로 정의되어 있는 오브젝트를 판단 후 해당 플레이어를 바라보게 구현

- 범위 안에 플레이어를 인식 후 상태창이 나오도록 구현

- 인식 범위에 랜덤 위치로 투명 오브젝트를 생성하고 오브젝트를 목표 이동하는 방식으로 동물의 이동을 구현

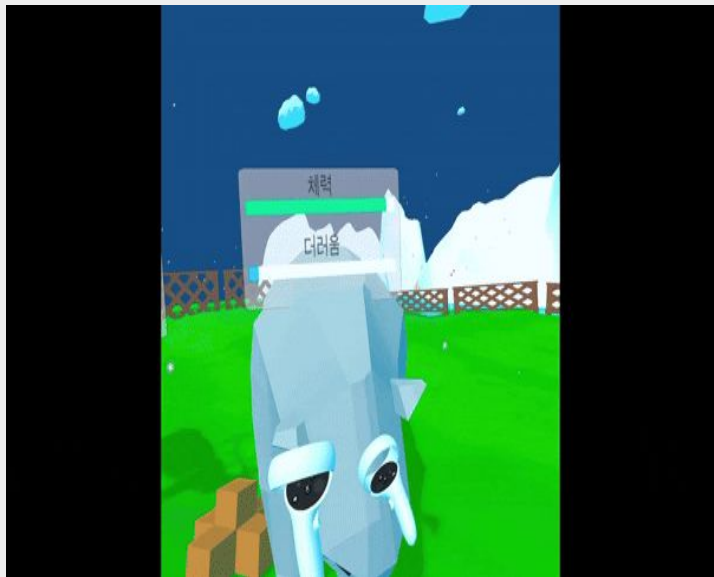
[2] - 상태창

- 상태창 구성으로 체력과 더러움이 바 형태로 UI이 제작

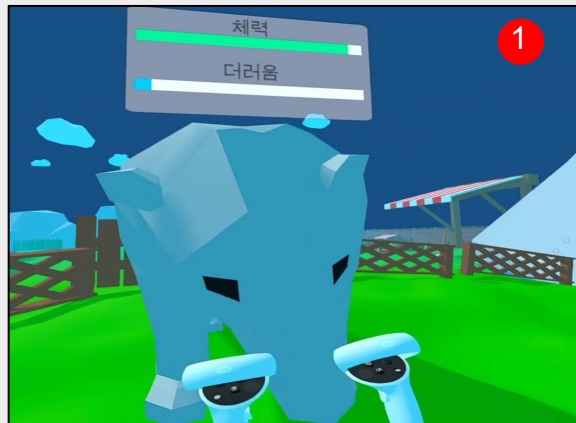
- 더러움이 높아지면 체력이 더 빨리 소모되며 체력이 다 소진 될 경우 동물은 Dead 상태로 변경

- 체력과 더러움 관리는 먹이와 목욕 상호작용으로 조절하도록 구현

시연



구성 요소

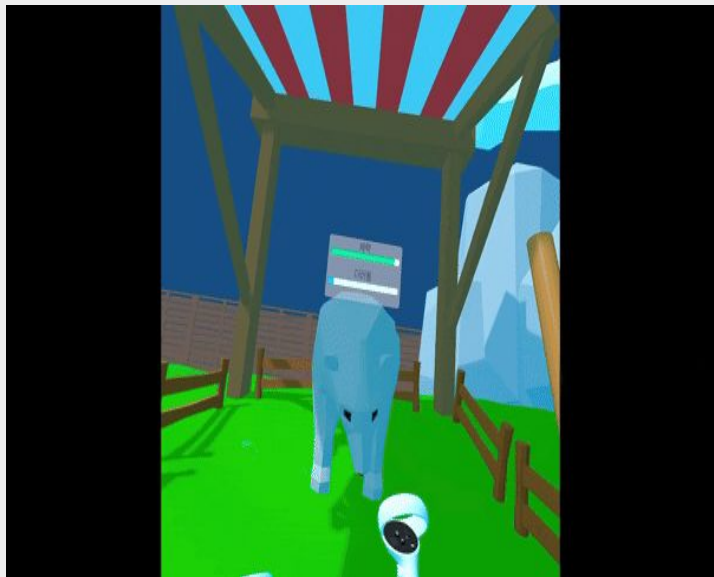


기능 설명

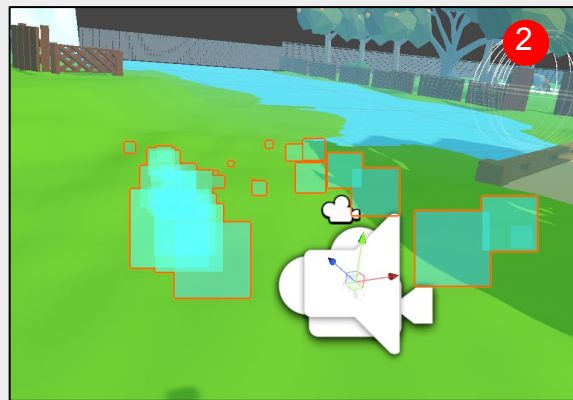
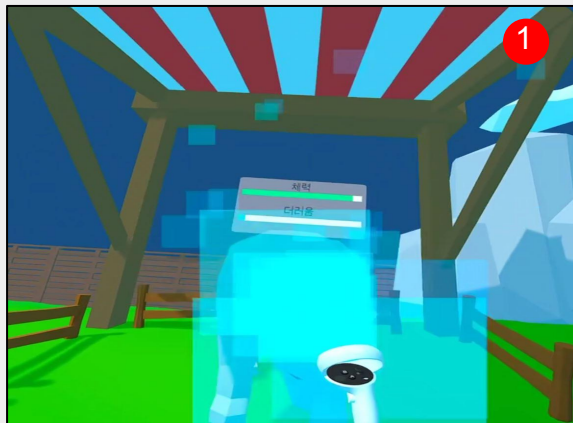
[1] - 물리 작용

- 각 오브젝트마다 Rigid Body(물리적 특성)를 설정하여 상호작용을 가능하도록 제작
- 각 상호작용으로 동물을 이동, 먹이 배급, 청결 관리 기능 설계

시연



구성 요소



기능 설명

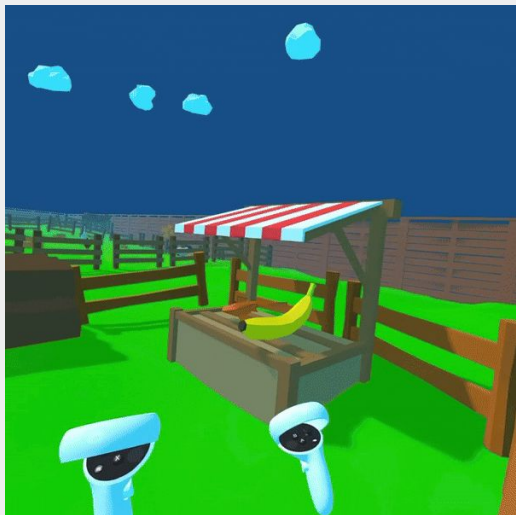
[1] - 동물 목욕

- 청소 전용 구역을 설정하여 구역에 동물이 도착하면 세척이 가능해 집니다.
- 동물을 목욕하면 동물의 더러움 정도를 관리할 수 있습니다.

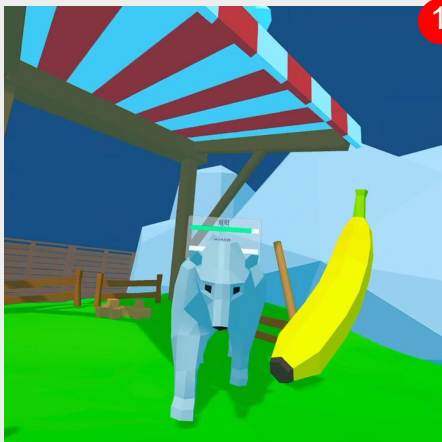
[2] 물 파티클

- 물 파티클을 제작하여 Oculus 오른쪽 기기 트리거를 사용하는 동안 파티클이 생성되도록 제작
- 동물 태그를 설정하고 동물 태그에 파티클이 접촉시 더러움을 제거하는 기능

시연



구성 요소



기능 설명

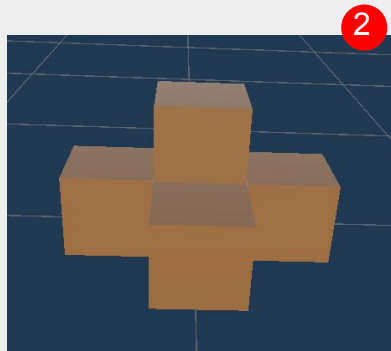
[1] - 먹이 배급

- 먹이 생성소에서 먹이를 가져와 동물에게 달으면 상태창에 체력이 회복 되도록 구현

시연



구성 요소



기능 설명

[1] - 청소 도구

- 빗자루 디자인으로 제작
- 솔 부분에 배변이 접촉 시 제거 되도록 기능 구현

[2] - 배변

- 동물마다 시간을 정해 배변 오브젝트를 자동으로 해당 위치에 생성 되도록 설정

프로젝트 시연 영상입니다.

PDF 사진 클릭시 Youtube 영상으로 넘어갑니다.



ISSUE

- 프로젝트 제작기간 : 23.09 ~ 23.12
- 팀원 : 고가희 / 강창민
- 담당 업무 : 회원가입, 공지사항, DashBoard 등 페이지 개발 및 Front & BackEnd 개발
- 사용 언어 : Java, JavaScript
- 툴 : MSSQL, Eclipse

테이블

USER

기능 설명

테이블 명	사용자 관리						
테이블 ID	SYS_USER	테이블 설명	사용자 관리 테이블				
작성자	강창민 / 고가희			작성일자	2023.09.03		
컬럼명	컬럼 ID	Type	Size	Not Null	PK	FK	비고
사용자 순번	USER_SEQ	BIGINT		V	V		
권한 순번	AUTH_SEQ	BIGINT				V	
사용자 아이디	USER_ID	VARCHAR	20	V			
사용자 명	USER_NM	VARCHAR	50	V			
비밀번호	PASSWORD	VARCHAR	50				
생년월일	BIRTH_DT	VARCHAR	20				19990726
핸드폰 번호	PHONE	VARCHAR	20				010XXXXXXXXX
상태	STATUS_CD	VARCHAR	20				
오류 개수	LOGIN_FAIL_CNT	INT					
작성자	REG_USER_SEQ	BIGINT					
작성 일자	REG_DT	DATETIME		V			
수정자	MODI_USRE_SEQ	BIGINT					
수정일자	MODI_DT	DATETIME					

사용자 순번 : 회원가입 시 유저의 고유 번호
권한 순번 : 현재 유저가 갖고 있는 권한 번호
사용자 아이디 : 회원가입시 입력 된 아이디 (필수 입력)
사용자 명 : 회원가입시 입력 된 이름 (필수 입력)
비밀번호 : 회원가입시 입력 된 비밀번호
생년월일 : 회원가입시 입력 된 생년월일
핸드폰 번호 : 회원가입시 입력 된 번호
상태 : 현재 업무투입 상태 확인
오류 개수 : 로그인 실패 횟수 확인

테이블 명	권한 관리						
테이블 ID	SYS_AUTH	테이블 설명	권한 관리 테이블				
작성자	강창민 / 고가희		작성일자	2023.09.03			
컬럼명	컬럼 ID	Type	Size	Not Null	PK	FK	비고
권한 순번	AUTH_SEQ	BIGINT		V	V		
권한 코드	AUTH_CD	VARCHAR	20				
권한 명	AUTH_NM	VARCHAR	50				
작성자	REG_USER_SEQ	BIGINT		V			
작성 일자	REG_DT	DATETIME		V			
수정자	MODI_USRE_SEQ	BIGINT					
수정일자	MODI_DT	DATETIME					

권한 순번 : 권한마다 고유 번호
 권한 코드 : User, Admin, View 권한 선택
 권한 명 : 권한 코드에 대한 권한 명

테이블

ITEM

기능 설명

테이블 명	조치사항 관리						
테이블 ID	GEM_ITEM	테이블 설명	조치사항 테이블				
작성자	강창민 / 고가희		작성일자	2023.09.03			
컬럼명	컬럼 ID	Type	Size	Not Null	PK	FK	비고
조치사항 순번	ITEM_SEQ	BIGINT		V	V		
조치사항 코드	ITEM_CD	VARCHAR	20	V			
조치사항 제목	ITEM_TITLE	VARCHAR	100				
조치사항 내용	ITEM_CONTENT	VARCHAR	MAX				
조치사항 타입	ITEM_TYPE	VARCHAR	20				오류/기능추가/...
상태 코드	STATUS_CD	VARCHAR	20				
기한 일자	ITEM_LIML_DT	DATETIME					
스텝	STEP	INT					
코드 순번	CD_SEQ	VARCHAR	20			V	메뉴 정보
작성자	REG_USER_SEQ	BIGINT		V			
작성 일자	REG_DT	DATETIME		V			
수정자	MODI_USRE_SEQ	BIGINT					
수정일자	MODI_DT	DATETIME					

조치사항 순번 : 조치사항 생성 시 고유 번호 할당
조치사항 코드 : 조치사항 고유 코드
조치사항 제목 : 조치사항 생성 시 제목 작성
조치사항 내용 : 요청자에게 전달 할 내용 작성
조치사항 타입 : 오류, 기능 추가 및 요구할 분야 선택
상태 코드 : 조치사항 처리 상태에 따라 변경(ex:처리중)
기한 일자 : 기한 설정
스텝 : 스텝을 통해 조치사항 순서 판별
코드 순번 : 오류가 발생한 화면 설정(ex:메인화면)

테이블

ITEM_REQUEST

테이블 명

조치사항 요청

테이블 ID

GEM_ITEM_REQUEST

테이블 설명

조치사항 요청 테이블

작성자

강창민 / 고가희

작성일자

2023.09.03

컬럼명

컬럼 ID

Type

Size

Not Null

PK

FK

비고

요청 순번

REQ_SEQ

BIGINT

V

V

조치사항 순번

ITEM_SEQ

BIGINT

V

V

요청자

REVIEW_USER_SEQ

BIGINT

처리자

OWNER_USER_SEQ

BIGINT

요구 제목

REQ_TITLE

VARCHAR

100

요구 내용

REQ_CONTENT

VARCHAR

MAX

처리 내용

REVIEW_CONTENT

VARCHAR

MAX

작성자

REG_USER_SEQ

BIGINT

V

작성 일자

REG_DT

DATETIME

V

수정자

MODI_USRE_SEQ

BIGINT

수정일자

MODI_DT

DATETIME

요청 순번

: 요청자에 고유 번호 생성

요청자

: 조치사항 생성한 사용자

처리자

: 요청자가 설정한 사용자

요구 제목

: 조치사항 작성 시 제목

요구 내용

: 요청자에게 전달할 내용

처리 내용

: 요구 사항에 대한 처리 내용

테이블

BOARD

기능 설명

테이블 명	공지사항 관리						
테이블 ID	SYS_BOARD	테이블 설명	공지사항 테이블				
작성자	강창민 / 고가희			작성일자	2023.09.21		
컬럼명	컬럼 ID	Type	Size	Not Null	PK	FK	비고
공지사항 순번	BOARD_SEQ	BIGINT		V	V		
제목	TITLE	VARCHAR	50				
내용	CONTENT_TXT	VARCHAR	MAX				
내용	CONTENT_HTML	VARCHAR	MAX				
정렬	SORT	INT					
사용 여부	VIEW_YN	CHAR	1				Y/N
조회수	VIEW_CNT	INT					
작성자	REG_USER_SEQ	BIGINT		V			
작성 일자	REG_DT	DATETIME		V			
수정자	MODI_USRE_SEQ	BIGINT					
수정일자	MODI_DT	DATETIME					

공지사항 순번 : 공지사항 생성 시 고유 번호

제목 : 공지사항 제목

내용(text) : 공지사항 내용 작성 시 TXT 형식으로 저장

내용(HTML) : 공지사항 내용 작성 시 HTML형식으로 저장

정렬 : 메인 화면 공지사항 우선순위 설정

사용 여부 : 사용할 공지사항을 선택

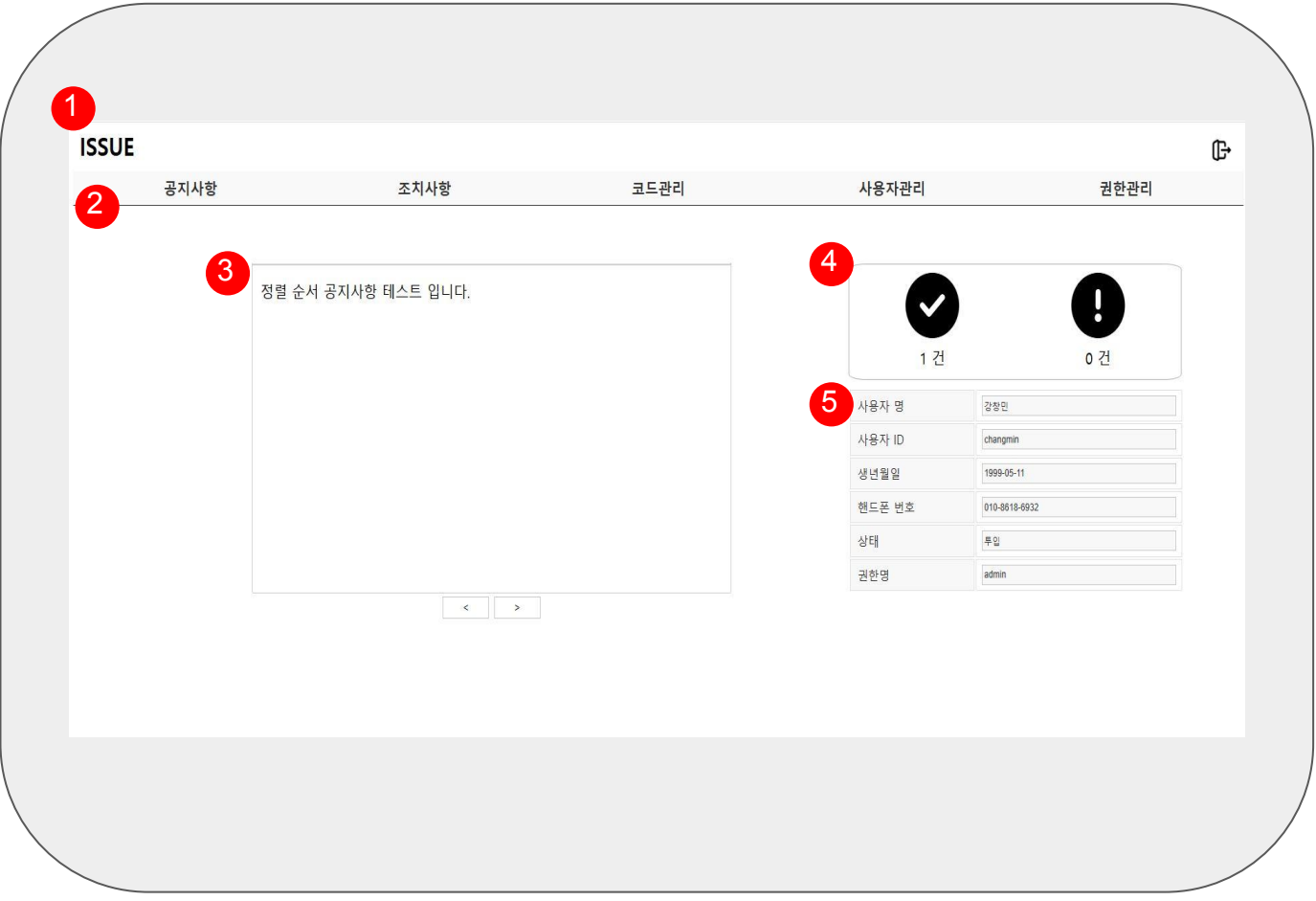
조회수 : 공지사항에 조회수 체크

화면명	로그인	메뉴 위치	로그인	기능 설명
The mockup shows a login interface with a large 'ISSUE' title. Below it are two input fields: the first is preceded by a person icon and labeled '아이디를 입력해주세요' (Please enter your ID); the second is preceded by a lock icon and labeled '비밀번호를 입력해주세요' (Please enter your password). A red circle with the number '1' points to a black 'LOGIN' button. Below the button, a red circle with the number '2' points to a purple link labeled '회원가입' (Sign Up).				사용자 ID와 PW 데이터베이스에서 대조하여 로그인 기능, 회원가입 팝업 생성하는 로그인 화면 [1 - Login 버튼] 로그인 버튼 클릭시 AJAX에서 url(loginUser.json)를 호출하여 RequestMapping을 통해 해당 url을 매핑하고 SQL 맵핑을 통해 해당 id를 찾고 SQL을 작동하여 작성한 아이디와 비밀번호를 확인하여 로그인 합니다. <pre>login.jsp(Ajax - url: /cmmn/login/loginUser.json) > LoginController.java(@RequestMapping(value="/cmmn/login/loginUser.json")) 매핑 이후 loginService.loginUser() 호출 > LoginService.java(comDao.select("cmmn_login.select_SY S_USER",mapSQL) 실행 > cmmn_login_SQL.xml(select id="select_SSYS_USER) SQL 문장 실행 후 다시 반환 하여 결과를 출력합니다.</pre> [2 - 회원가입] 회원가입 팝업창을 생성

화면명	로그인	메뉴 위치	로그인	기능 설명
			1 <pre>\$.ajax({ type: 'POST' ,async: true ,url: '/cmmn/login/loginUser.json' ,dataType: 'json' ,data: { user_id: \$("#user_id").val() ,user_pw: \$("#user_pw").val() } })</pre> 3 <pre>@SuppressWarnings("unused") public HashMap<String, Object> loginUser(HashMap<String, Object> reqMap) throws Exception { HashMap<String, Object> mapSQL = new HashMap<String, Object>(); mapSQL.put("user_id", reqMap.get("user_id")); HashMap<String, Object> userInfo = comDao.select("cmmn_login.select_SYS_USER", mapSQL);</pre> 4 <pre><select id="select_SYS_USER" parameterType="hashmap" resultType="hashmap"> SELECT * FROM SYS_USER WHERE 1=1 <if test='user_seq != "" and user_seq != null'> AND USER_SEQ = #{user_seq} </if> <if test='user_id != "" and user_id != null'> AND USER_ID = #{user_id} </if> </select></pre>	사용자 ID와 PW 데이터베이스에서 대조하여 로그인 기능, 회원가입 팝업 생성하는 로그인 화면 [1] login.jsp(Ajax - url: /cmmn/login/loginUser.json) 입력한 id, pw를 서버에 보냅니다. [2] LoginController.java(@RequestMapping(value="/cmmn/login/loginUser.json") loginUser() loginUser()) url 매핑으로 메서드를 찾고 loginUser를 호출합니다. [3] LoginService.java(comDao.select("cmmn_login.select_SYS_USER",mapSQL)) SQL 매핑으로 namespace : cmmn_login / id : select_SYS_USER SQL 실행 [4] cmmn_login_SQL.xml(select id="select_SYS_USER) 해당 SQL을 실행하여 결과를 전달합니다.

화면명	회원가입	메뉴 위치	Login > 회원가입	기능 설명
1 회원가입  아이디를 입력해주세요  비밀번호를 입력해주세요 2  사용자 이름  생년월일 (ex. 19990101)  핸드폰 번호 (ex. 010-1111-1111) 3 요청				사용자는 회원가입을 통해 시스템에 등록을 한다. [1 - Validation] 아이디 중복 체크 비밀번호 유효성 체크 [2 - 비밀번호 유효성 확인] 비밀번호 양식 표시 [3 - 요청] 시스템 관리자에게 회원가입 요청

화면명	회원가입	메뉴 위치	Login > 회원가입	기능 설명
				사용자는 회원가입을 통해 시스템에 등록을 한다. [1] Ajax - url: /sys/user/userSave.json 입력한 회원 정보를 서버에 보냅니다. [2] @RequestMapping(value = "/sys/user/userSave.json", method = {RequestMethod.POST, RequestMethod.GET}) url 매핑으로 메서드를 찾고 userSave()를 호출합니다. [3] SQL 매핑으로 namespace : sys_user / id : insert_SYS_USER SQL 실행 [4] cmnn_login_SQL.xml(select id="select_SYS_USER) 해당 SQL을 실행하여 데이터베이스에 정보를 저장합니다..

화면명	메인화면	메뉴 위치	메인화면	기능 설명												
 The screenshot shows a web application dashboard. Callout 1 points to the 'ISSUE' title. Callout 2 points to the navigation tabs: '공지사항' (Notice), '조치사항' (Action), '코드관리' (Code Management), '사용자관리' (User Management), and '권한관리' (Permission Management). Callout 3 points to a notice area containing the text '정렬 순서 공지사항 테스트 입니다.' (Sorting order notice test). Callout 4 points to a summary box showing '1 건' (1 item) with a checkmark icon and '0 건' (0 items) with an exclamation mark icon. Callout 5 points to a user information table. <table><tr><td>사용자 명</td><td>강창민</td></tr><tr><td>사용자 ID</td><td>changmin</td></tr><tr><td>생년월일</td><td>1999-05-11</td></tr><tr><td>핸드폰 번호</td><td>010-8818-0932</td></tr><tr><td>상태</td><td>투입</td></tr><tr><td>권한명</td><td>admin</td></tr></table>				사용자 명	강창민	사용자 ID	changmin	생년월일	1999-05-11	핸드폰 번호	010-8818-0932	상태	투입	권한명	admin	Main(Dashboard) 화면으로 해당 사용자 관련 정보를 간단하게 확인할 수 있다. [1 - ISSUE 아이콘] 아이콘을 클릭하면 Main(Dashboard)로 페이지 이동 [2 - 메뉴 테이블] 클릭 이벤트를 통해 각 해당 페이지로 이동 [3 - 시스템 공지사항] CKEditor를 통해 제작된 공지사항을 우선 순위에 따라 제공 [4 - 사용자 활동 정보] 사용자의 조치사항 해결 건 수를 표시 조치사항 해결 중인 문제 건 수를 표시 [5 - 사용자 기본 정보 제공] 사용자 정보를 데이터베이스에서 가져와 표시
사용자 명	강창민															
사용자 ID	changmin															
생년월일	1999-05-11															
핸드폰 번호	010-8818-0932															
상태	투입															
권한명	admin															

화면명	메인화면	메뉴 위치	메인화면	기능 설명
			<pre> <mapper namespace="cmmn_main"> <select id="select_SYS_BOARD" parameterType="hashmap" resultType="hashmap"> SELECT CONTENT_HTML FROM SYS_BOARD WHERE VIEW_YN = 'Y' ORDER BY SORT ASC, ISNULL(MODI_DT, REG_DT) ASC </select> </pre> <pre> <select id="select_SYS_USER" parameterType="hashmap" resultType="hashmap"> SELECT USER_ID ,USER_NM ,SUBSTRING(PHONE,0,4) + '-' + SUBSTRING(PHONE,4,4) + '-' + SUBSTRING(PHONE,8,4) AS PHONE ,CONVERT(DATE, BIRTH_DT, 105) AS BIRTH_DT ,STATUS_CD ,CASE STATUS_CD WHEN 'DF' THEN '요청' WHEN 'IN' THEN '투입' END AS STATUS_NM ,(SELECT AUTH_NM FROM SYS_AUTH WHERE AUTH_SEQ = SYS_USER.AUTH_SEQ) AS AUTH_NM FROM SYS_USER WHERE USER_SEQ = #{login_user_seq} </select> </pre> <pre> <select id="select_GEM_ITEM" parameterType="hashmap" resultType="hashmap"> SELECT 'IN' ITEM_TYPE ,ISNULL((SELECT COUNT(1) CNT FROM GEM_ITEM ITEM LEFT JOIN GEM_ITEM_REQUEST REQ ON ITEM.ITEM_SEQ = REQ.ITEM_SEQ WHERE (REVIEW_USER_SEQ = #{login_user_seq} OR OWNER_USER_SEQ = #{login_user_seq}) AND ITEM_STATUS_CD IN ('RC', 'RV', 'DI') GROUP BY ITEM_STATUS_CD), 0) CNT UNION ALL SELECT 'CP' ITEM_TYPE ,ISNULL((SELECT COUNT(1) CNT FROM GEM_ITEM ITEM LEFT JOIN GEM_ITEM_REQUEST REQ ON ITEM.ITEM_SEQ = REQ.ITEM_SEQ WHERE (REVIEW_USER_SEQ = #{login_user_seq} OR OWNER_USER_SEQ = #{login_user_seq}) AND ITEM_STATUS_CD IN ('CP') GROUP BY ITEM_STATUS_CD), 0) CNT </select> </pre>	사용자는 회원가입을 통해 시스템에 등록을 한다. [1] select_SYS_BOARD를 통해 사용여부가 활성화 되어있는 공지사항 중 정렬 컬럼(sort)을 오름차순으로 정렬하며 MODI_DT가 NULL이면 REG_DT를 사용하여 날짜가 최신순으로 메인 화면 공지사항에 나오게 SQL을 작성 [2] select_SYS_USER를 통해 데이터베이스 USER 테이블에서 사용자의 데이터를 SELECT하여 가져와 표에 시각화 [3] 위 SELECT문은 현재 사용자가 REVIEW_USER_SEQ(처리자) or OWNER_USER_SEQ(요청자) 상태에 포함 되어있으며 RC(요청 확인), RV(리뷰 요청), DI(개발 진행) 상태인 조치사항 수를 카운트하여 메인화면에 표시 아래 SELECT 문은 CP(완료) 상태에 조치사항 수를 카운트하여 메인화면에 표시

ISSUE

공지사항

조치사항

코드관리

사용자관리

권한관리

	내용	제목	작성자	날짜	정렬
1	공지사항 테스트	정렬 순서 공지사항 테스트입니다.	강창민	2025-02-16 0...	0
2	TEST 2	TEST 2TEST 2	강창민	2025-02-16 0...	1
3	TEST 3	TEST 3TEST 3	강창민	2025-02-16 0...	3
4	TEST 4	TEST 4	강창민	2025-02-16 0...	2

4

제목

작성자

날짜

정렬

사용여부

공지사항 테스트

강창민

2025-02-16 06:46

0

☒

↶ ↷ 문단

B *I*

정렬 순서 공지사항 테스트입니다.

1

2

3

1/4 <> 페이지 1 / 1 >> > 10 >

시스템 관리자는 공지사항을 등록 / 수정 / 삭제 할 수 있습니다.

[1 - Add]

Clear 기능을 구현하여 작성한 내용을 초기화

[2 - Delete]

선택한 공지사항을 시스템에서 삭제

[3 - Save]

작성한 공지사항을 시스템에 등록

[4 - 공지사항 제작]

CKEditor, 테이블을 이용하여 작성한 내용을 시스템에 저장

사용여부 및 정렬 순서를 통해 Main(Dashboard) 화면에 제공

화면명	공지사항	메뉴 위치	메인화면 > 공지사항	기능 설명
				공지사항 저장 기능에 대한 설명입니다. [1] url: '/sys/notice/noticeSave.json?'+params 입력한 공지사항 정보를 서버에 보냅니다. CKEditor HTML 정보를 저장하기 위해 TXT형태로 변형 후 서버로 데이터를 전달 [2] @RequestMapping(value = "/sys/notice/noticeSave.json", method = {RequestMethod.POST, RequestMethod.GET}) loginService.loginUser() noticeService.noticeSave(reqMap); url 매핑으로 메서드를 찾고 noticeSave()를 호출합니다. [3] public HashMap<String, Object> noticeSave(HashMap<String, Object> reqMap) throws Exception { SaveType으로 sql insert, update, delete를 구분하였고 신규 공지사항을 저장하기에 I(insert)를 활용해 코드 실행 SQL 매핑으로 namespace : sys_board / id : insert_SYS_BOARD SQL 실행 [4] sys_notice_SQL.xml(select id="insert_SYS_BOARD) 해당 SQL을 실행하여 결과를 데이터베이스에 전달합니다..

ISSUE

공지사항

조치사항

코드관리

사용자관리

권한관리

Issue Code	메뉴	제목	상태	요청자	요청 내용	처리자	처리 내용
1	20231212.1	Dashboard	완료	TestUser	테스트입니다.	김영민	다시 확인합니다.
2	20230217.1	권한관리	완료	김영민	테스트 권한이 제대로 작동이 안됩니다.	최소희	SQL문 수정하여 해결하였습니다.
3	20230217.2	공지사항	완료	김영민	공지사항 설정이 불가능합니다.	최소희	코드 수정하여 해결하였습니다.
4	20230224.1	Dashboard	완료	김영민	테스트입니다.	최소희	다시 확인합니다.
5	20230224.2	Dashboard	완료	김영민	테스트입니다.	최소희	다시 확인합니다.

3

localhost:3080/gem/issue/issueRegistrationPopup.view?item_seq=1 - Chrome

localhost:3080/gem/issue/issueRegistrationPopup.view?item_seq=1

전송

요청자

Item Code

상태

동시저장

* Type

요류

* 메뉴

Dashboard

* 제목

요청 내용

처리자

최소희 (test)

* 처리 기한

요청자

작성일자

처리자

처리 내용

요청자 Review

완료일자

4

5

관리자는 조치사항을 처리자를 지정하여 요청할 수 있습니다.

[요청자 시점]

[1 - Add]

새로운 조치사항 팝업을 생성

[2 - Delete]

선택한 조치사항을 시스템에서 삭제

[3 - 조치사항 팝업창]

Type : select box로 구성, 옵션값으로 오류, 기능수정, 기능추가, 질문 구성 및 선택

메뉴 : select box로 구성, 각 화면명으로 옵션 구성
제목, 요청 내용 : 전달하고자 하는 내용을 작성

처리자 : select box로 구성, 투입(IN) 상태 인원으로 옵션 설정 및 선택

처리 기한 : Datepicker를 사용하여 날짜를 설정

상태 : 조치사항 처리 과정에 따라 (임시 저장, 요청 확인중, 개발 진행중, 테스트 요청, 완료) 상태 순으로 변경

요청자 : 현재 로그인한 계정으로 설정

작성일자 : 현재 시간 전송 및 저장 시 현재 시간으로 설정

[4 - 전송]

작성한 요구사항 내용을 처리자에게 전송
메시지 상태가 요청 확인중으로 변경

[5 - Save]

작성한 내용을 임시 저장

1

```
function setting() {
  //
  Step = '1' : Creator 임시 제공
  Step = '2' : 요청 확인
  Step = '3' : 개발 진행
  Step = '4' : 테스트 진행
  Step = '5' : 개발 완료
  //
  if (item_seq == "" || item_seq == -1 || ($("req_user_seq").val() == $("login_user_seq").val()
    auth = 'creator';
  } else if ($("owner_user_seq").val() == $("login_user_seq").val() {
    auth = 'owner';
  }

  let step = $("step").val();
  let setting = false;

  $("rejectBtn").attr('hidden', true);
  $("sendBtn").attr('hidden', true);
  $("saveBtn").attr('hidden', true);

  $("item_limi_dt").attr('disabled', true);
  $("item_type").attr('disabled', true);
  $("item_menu").attr('disabled', true);
  $("item_title").attr('disabled', true);
  $("item_content").attr('disabled', true);
  $("owner_user_seq").attr('disabled', true);
  $("req_content").attr('disabled', true);
  $("review_content").attr('disabled', true);

  if (auth == 'creator') {
    if (step == "" || step == '1' || step == '2') {
      $("sendBtn").attr('hidden', false);
      $("saveBtn").attr('hidden', false);

      $("item_limi_dt").attr('disabled', false);
      $("item_type").attr('disabled', false);
      $("item_menu").attr('disabled', false);
      $("item_title").attr('disabled', false);
      $("item_content").attr('disabled', false);
      $("owner_user_seq").attr('disabled', false);
    }
  }
}
```

2

```
$.ajax({
  type: 'POST',
  async: true
  ,url: '/gem/issue/issueSave.json'
  ,dataType: 'json'
  ,data: params
  ,error:function (data, textStatus) {
    alert("시스템 에러입니다.");
  }
});
```

3

```
@RequestMapping(value = "/gem/issue/issueSave.json", method = {RequestMethod.POST, RequestMethod.GET})
public String 조치사항저장(@RequestParam HashMap<String, Object> reqMap, ModelMap model) throws Exception {
  HashMap<String, Object> result = issueService.issueSave(reqMap);
}
```

4

```
public HashMap<String, Object> issueSave(HashMap<String, Object> reqMap) throws Exception {
  String saveType = "U";

  // Insert인지 Update인지 확인
  if (reqMap.get("item_seq").equals("")) {
    saveType = "I";
  }

  if (saveType == "I") {
    reqMap.put("item_seq", this.getItemSeq(reqMap)); // SEQ 부여
    reqMap.put("item_cd", this.getItemCd(reqMap)); // Code 부여
  }

  if (saveType == "I") {
    comDao.insert("gem_issue.insert_GEM_ITEM", reqMap);
  }

  if (reqMap.get("owner_user_seq").equals("") == false) {
    comDao.insert("gem_issue.merge_GEM_ITEM_REQUEST", reqMap);
  }
}
```

5

```
update id="merge_GEM_ITEM_REQUEST" parameterType="hashmap">
  IF (SELECT COUNT(1) FROM GEM_ITEM_REQUEST WHERE REQ_ITEM_SEQ = #{item_seq} AND REQ_REQ_SEQ = #{req_seq}) > 0 BEGIN
    INSERT INTO GEM_ITEM_REQUEST
      (REQ_SEQ, ITEM_SEQ, OWNER_USER_SEQ, REQ_CONTENT
      , REVIEW_USER_SEQ, REVIEW_CONTENT, REQ_USER_SEQ, REQ_DT)
    SELECT NEXT VALUE FOR GEM_ITEM_REQUEST_SEQ
      ,#{item_seq}
      ,#{owner_user_seq}
      ,#{req_content}
      ,#{login_user_seq}
      ,#{review_content}
      ,#{login_user_seq}
      ,GETDATE();
  END
  ELSE BEGIN
    UPDATE GEM_ITEM_REQUEST
    SET
      OWNER_USER_SEQ = #{owner_user_seq}
      ,REQ_CONTENT = #{req_content}
      ,REVIEW_USER_SEQ = #{review_user_seq}
      ,REVIEW_CONTENT = #{review_content}
      ,REQ_USER_SEQ = #{login_user_seq}
      ,REQ_DT = GETDATE()
    WHERE REQ_SEQ = #{req_seq}
  END
</update>

<insert id="insert_GEM_ITEM" parameterType="hashmap">
  INSERT INTO GEM_ITEM (
    ITEM_SEQ
    , ITEM_CD
    , ITEM_TITLE
    , ITEM_CONTENT
    , ITEM_TYPE
    , ITEM_MENU
    , ITEM_STATUS_CD
    <if test="item_limi_dt != '' and item_limi_dt != null">
      , ITEM_LIMI_DT
    </if>
    , STEP
    , REQ_USER_SEQ
    , REQ_DT
  )
  SELECT
    #{item_seq}
    , #{item_cd}
    , #{item_title}
    , #{item_content}
    , #{item_type}
    , #{item_menu}
    , #{item_status_cd}
    <if test="item_limi_dt != '' and item_limi_dt != null">
      , #{item_limi_dt}
    </if>
    , #{step}
    , #{login_user_seq}
    , GETDATE()
  </insert>
```

관리자는 조치사항을 처리자를 지정하여 요청할 수 있습니다.

1)

조치사항 팝업을 생성할 때 처리자인지 새로 작성자인지 판단하여 creator와 Step을 if문을 통해 확인하여 type, 메뉴, 요청 내용 등 disabled 타입으로 변경

2)

조치사항을 작성 후 전송 버튼 활성화 후 상태가 개발진행중 상태로 바뀌고 step 별류가 3로 바뀌며 AJAX 가 실행 되어 '/gem/issue/issueSave.json' url를 매핑

3)

issueSave() 메서드를 실행하여 조치사항에 입력 정보를 메서드에 전달합니다.

4)

처음 생성하기 때문에 item_seq 가 "" 으로 설정되어 있기에 saveType이 I 로 설정되며 item_seq와 item_cd 부여

comDao.insert("gem_issue.insert_GEM_ITEM", reqMap);
comDao.insert("gem_issue.merge_GEM_ITEM_REQUEST", reqMap);
실행합니다.

5)

<insert id="insert_GEM_ITEM" parameterType="hashmap">

<update id="merge_GEM_ITEM_REQUEST" parameterType="hashmap">

sql을 실행하여 GEM_ITEM 테이블과 GEM_ITEM_REQUEST 테이블에 작성한 정보를 저장합니다.

ISSUE

공지사항

조치사항

코드관리

사용자관리

권한관리

Issue Code	메뉴	제목	상태	요청자	요청 내용	처리자	처리 내용
1	Dashboard	test 입니다	완료	Test_user	테스트 입니다	강형민	다시 확인했습니다.
2	권한관리	권한 관리 테스트 권한이 오류	완료	강형민	테스트 권한이 제대로 작동이 안됩니다.	테스트계정	SQL문 수정하여 해결하였습니다.
3	공지사항	공지사항 생성이 원활하지 않습니다.	개발 진행중	강형민	공지사항 생성이 불가능합니다.	테스트계정	

1

localhost:8080/gem/issue/issueRegistrationPopup.view?item_seq=53 - Chrome

localhost:8080/gem/issue/issueRegistrationPopup.view?item_seq=53

전송

요청자

Item Code	20250217_2	상태	개발 진행중
* Type	오류	* 메뉴	공지사항
* 제목	공지사항 생성이 원활하지 않습니다.		
요청 내용	공지사항 생성이 불가능합니다.		
처리자	테스트계정 (test2)	* 처리 기한	2025-02-28 00:00
요청자	강형민	작성일자	2025-02-17 01:54

처리자

처리 내용	요드 수정하여 해결했습니다.
요청자 Review	
완료 일자	

요청자에게 조치사항을 받은 처리자는 요청 내용을 확인하고 처리한 내용을 작성.

[처리자 시점]

[1 - 처리자 팝업]

요청자가 조치사항을 보내면 상태는 개발 진행중으로 변경이 되며 처리 내용 입력칸이 활성화 됩니다.

처리자는 요청 내용을 확인하며 문제를 해결하고 해결 방법 및 처리 내용을 입력하여 전송 버튼을 눌러 요청자에게 다시 전달합니다.

```
function settingUI() {
    /*
    Step = '1' : Creator 임시 저장
    Step = '2' : 요청 확인
    Step = '3' : 개발 진행
    Step = '4' : 테스트 진행
    Step = '5' : 개발 완료
    */
    if (item_seq == '' || item_seq == -1 || S("#req_user_seq").val() == S("#login_user_seq")) {
        auth = 'creator';
    } else if (S("#owner_user_seq").val() == S("#login_user_seq")) {
        auth = 'owner';
    }

    let step = S("#step").val();
    let setflag = false;

    else if (auth == 'owner') {
        if (step == '3') {
            S("#sendBtn").attr('hidden', false);
            S("#saveBtn").attr('hidden', false);

            S("#req_content").attr('disabled', false);
        }
    }
}
```

```
@RequestMapping(value = "/gem/issue/issueSave.json", method = {RequestMethod.POST, RequestMethod.GET})
public String 조치사항저장(@RequestParam HashMap<String, Object> reqMap, Model<Map> model) throws Exception {
    HashMap<String, Object> result = issueService.issueSave(reqMap);
    model.addAttribute("data", result);
    return "jsonTemplate";
}

public HashMap<String, Object> issueSave(HashMap<String, Object> reqMap) throws Exception {
    String saveType = "U";

    // Insert인지 Update인지 확인
    if (reqMap.get("item_seq").equals("")) {
        saveType = "I";
    }

    if (saveType == "I") {
        reqMap.put("item_seq", this.getItemSeq(reqMap)); // SEQ 번호
        reqMap.put("item_cd", this.getItemCd(reqMap)); // Code 번호
    }

    if (saveType == "I") {
        comDao.insert("gem_issue.insert_GEM_ITEM", reqMap);
    } else if (saveType == "U") {
        comDao.insert("gem_issue.update_GEM_ITEM", reqMap);
    }

    if (reqMap.get("owner_user_seq").equals("")) == false {
        comDao.insert("gem_issue.merge_GEM_ITEM_REQUEST", reqMap);
    }

    return reqMap;
}
```

```
S("#sendBtn").click(function(){
    // Validation
    // (1/2) 필수 값 체크
    if (S("#item_title").val() == "") {
        alert("제목을 입력해주세요.");
        S("#item_title").focus();
        return;
    }

    // (1/2) 처리기한 체크
    if (new Date(S("#item_liml_dt").val()) < new Date()) {
        alert("처리 기한이 작성일보다 작습니다.");
        S("#item_liml_dt").focus();
        return;
    }

    saveData('send');
});
```

```
<update id="update_GEM_ITEM" parameterType="hashmap">
    UPDATE GEM_ITEM
    SET
        ITEM_TITLE = #{item_title}
        , ITEM_CONTENT = #{item_content}
        , ITEM_TYPE = #{item_type}
        , ITEM_MENU = #{item_menu}
        , ITEM_STATUS_CD = #{item_status_cd}
        <if test='item_liml_dt != "" and item_liml_dt != null'>
            , ITEM_LIML_DT = #{item_liml_dt}
        </if>
        , STEP = #{step}
        , MODI_USER_SEQ = #{login_user_seq}
        , MODI_DT = GETDATE()
    WHERE ITEM_SEQ = #{item_seq}
</update>
```

```
update (select count(*) from gem_item_request req where req.item_seq = A(item_seq) and req.req_seq = B(req_seq)) + B
insert into gem_item_request
(item_seq, item_seq, owner_user_seq, req_content,
 review_user_seq, review_content, req_user_seq, req_dt)
select req.item_seq, req.item_seq, req.owner_user_seq,
A(req_content),
A(req_user_seq),
A(review_content),
A(req_user_seq),
req_dt
from (
select req.item_seq, req.item_seq, req.owner_user_seq,
req.req_seq, req.req_seq, req.req_content,
req.req_user_seq, req.req_dt
from gem_item_request req
where req.item_seq = A(item_seq)
and req.req_seq = B(req_seq))

```

```
} else if (auth == 'owner') {
    if (btnType == 'send') {
        S("#item_status_cd").val("RV");
        S("#step").val("4");
    }
}

function saveData(btnType) {
    step = S("#step").val();
    let target = S("#dataForm");
    let disabled = target.find("disabled").removeAttr("disabled");
    let param = target.serialize() + "&login_user_seq=" + S("#login_user_seq").val() + "&item_seq=" + S("#item_seq").val();
    disabled.attr("disabled", "disabled");
    $.ajax({
        type: "POST",
        url: "/gem/issue/issueSave.json",
        dataType: "json",
        data: param,
        error: function (data, textStatus) {
            alert("시스템 에러입니다.");
        },
        success: function (data, textStatus) {
            alert("저장되었습니다.");
            item_seq = data.data.item_seq;
            location.href();
            window.open(issueSearch());
        }
    });
}
```

요청자에게 조치사항을 받은 처리자는 요청 내용을 확인하고 처리한 내용을 작성.

[1]
owner_user_seq 와 login_user_seq 가 동일하기 때문에 auth = 'owner'로 변경되고 처리 내용 작성칸이 활성화로 변경

[2]
처리 내용을 작성후 전송 버튼 클릭시 발생 이벤트

[3]
상태를 테스트 요청 상태로 변경하고 스텝을 4(테스트 진행)으로 변경

"/gem/issue/issueSave.json" url 매핑

[4]
issueService.issueSave(reqMap) 메서드를 실행

saveType = 'U' 이므로
comDao.insert("gem_issue.update_GEM_ITEM", reqMap)
comDao.insert("gem_issue.merge_GEM_ITEM_REQUEST", reqMap) 두 SQL 매핑이 실행

[5]
ITEM테이블에 UPDATE문 실행 후 ITEM_STATUS_CD, STEP,, 수정 시간이 업데이트 됐으며 ITEM_REQUEST 테이블에는 REVIEW_CONTENT 갱신

ISSUE

공지사항

조치사항

코드관리

사용자관리

권한관리

	Issue Code	메뉴	제목	상태	요청자	요청 내용	처리자	처리 내용
1	20231212_1	Dashboard	test 입니다	완료	Test_user	타스트 입니다.	강창민	다시 확인했습니다.
2	20250217_1	권한관리	권한 관리 테스트 권한이 오류	완료	강창민	테스트 권한이 제대로 작동이 안됩니다.	테스트계정	SQL문 수정하여 해결하였습니다.
3	20250217_2	공지사항	공지사항 생성이 원활하지 않습니다.	테스트 요청	강창민	공지사항 생성이 불가능합니다.	테스트계정	코드 수정하여 해결했습니다.

1

2

3

요청자

Item Code

20250217_2

상태

테스트 요청

* Type

오류

* 메뉴

공지사항

* 제목

공지사항 생성이 원활하지 않습니다.

요청 내용

공지사항 생성이 불가능합니다.

처리자

테스트계정 (test2)

* 처리 기한

2025-02-28 00:00

요청자

강창민

작성일자

2025-02-17 01:54

처리자

처리 내용

코드 수정하여 해결했습니다.

요청자 Review

해 확인했습니다

완료일자

처리자의 내용을 확인하고 최종 Review를 작성 후 반려, 완료, 임시 저장이 가능하도록 구현

[요청자 시점]

[1 - 반려]

처리자의 처리 내용 리뷰를 확인

처리 내용에 대해 수정 사항 요청자 review를 작성 후 처리자에게 메시지 전송

[2 - 완료]

요청 내용에 대해 처리 완료 확인

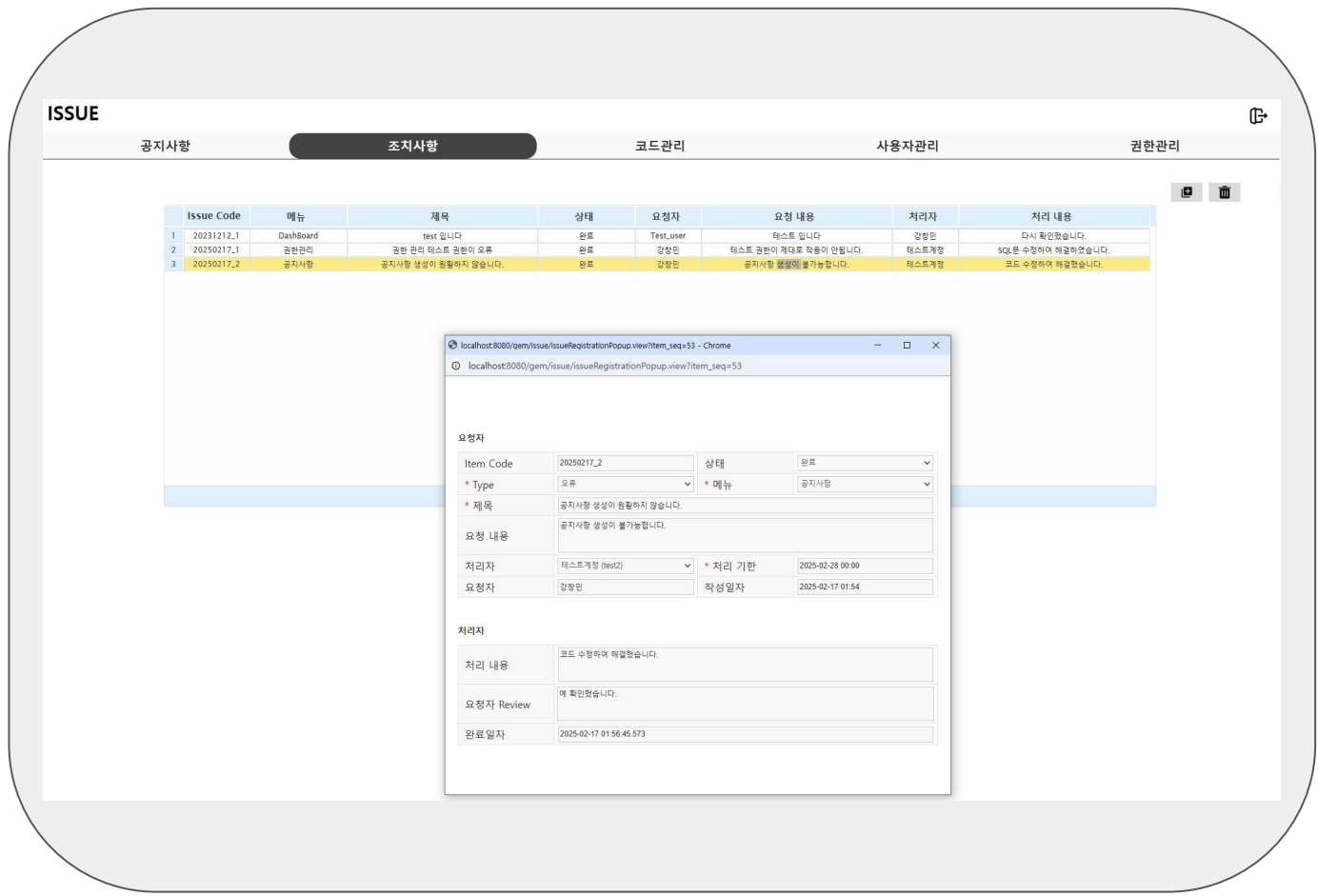
요청자 Review를 작성

완료일자에 현재 시각이 설정되고 조치사항이 저장

[3 - 저장]

작성한 메시지를 임시 저장

조치사항이 최종 완료 되었을 때 상태가 완료 상태로 변경되고 모든 처리가 완료된 상태



화면명	사용자관리	메뉴 위치	메인화면 > 사용자 관리	기능 설명																																										
ISSUE 공지사항 조치사항 코드관리 사용자관리 권한관리 5 <table><thead><tr><th></th><th>이름</th><th>아이디</th><th>생년월일</th><th>핸드폰</th><th>권한명</th><th>상태</th></tr></thead><tbody><tr><td>1</td><td>장민</td><td>test2</td><td></td><td>--</td><td>admin</td><td>IN</td></tr><tr><td>2</td><td>고가희</td><td>kahee</td><td>1999-07-26</td><td>010-2306-9736</td><td>view</td><td>IN</td></tr><tr><td>3</td><td>강장민</td><td>changmin</td><td>1999-05-11</td><td>010-8618-6932</td><td>admin</td><td>IN</td></tr><tr><td>4</td><td>Test_user</td><td>test</td><td>9999-05-11</td><td>111-1111-1111</td><td>view</td><td>IN</td></tr><tr><td>5</td><td>1234</td><td>12345</td><td></td><td>--</td><td></td><td>DF</td></tr></tbody></table> 4 이름 강장민 아이디 changmin 비밀번호 1234 생년월일 1999-05-11 핸드폰 번호 010-8618-6932 상태 투입 권한명 admin (ADMIN) 1 2 3 1 - User Add 필수 : 사용자 명, 사용자 ID, 비밀번호 Validation : 회원가입 화면과 동일 입력한 데이터를 초기화 초기 상태로 변경 2 - User Delete 선택한 사용자를 시스템에서 제거 3 - User Save 작성 및 수정 한 User Data를 시스템에 저장 4 - User Info User List에서 선택한 사용자 정보 제공 선택 사용자 정보 수정 가능 (선택시 사용자 ID는 Read Only) 5 - User List 모든 사용자가 리스트에 표시 사용자 이름 순으로 정렬					이름	아이디	생년월일	핸드폰	권한명	상태	1	장민	test2		--	admin	IN	2	고가희	kahee	1999-07-26	010-2306-9736	view	IN	3	강장민	changmin	1999-05-11	010-8618-6932	admin	IN	4	Test_user	test	9999-05-11	111-1111-1111	view	IN	5	1234	12345		--		DF	시스템 관리자는 사용자 정보 및 사용자 권한, 상태를 수정 필수 : 사용자 명, 사용자 ID, 비밀번호 Validation : 회원가입 화면과 동일 입력한 데이터를 초기화 초기 상태로 변경 2 - User Delete 선택한 사용자를 시스템에서 제거 3 - User Save 작성 및 수정 한 User Data를 시스템에 저장 4 - User Info User List에서 선택한 사용자 정보 제공 선택 사용자 정보 수정 가능 (선택시 사용자 ID는 Read Only) 5 - User List 모든 사용자가 리스트에 표시 사용자 이름 순으로 정렬
	이름	아이디	생년월일	핸드폰	권한명	상태																																								
1	장민	test2		--	admin	IN																																								
2	고가희	kahee	1999-07-26	010-2306-9736	view	IN																																								
3	강장민	changmin	1999-05-11	010-8618-6932	admin	IN																																								
4	Test_user	test	9999-05-11	111-1111-1111	view	IN																																								
5	1234	12345		--		DF																																								

화면명	권한관리	메뉴 위치	권한관리	기능 설명																													
ISSUE 공지사항 조치사항 코드관리 사용자관리 권한관리 1 2 3 4 5 <table><thead><tr><th></th><th>권한코드</th><th>권한명</th><th>작성자</th><th>작성일자</th></tr></thead><tbody><tr><td>1</td><td>USER</td><td>user</td><td>강형민</td><td>2023-12-12 14:31</td></tr><tr><td>2</td><td>ADMIN</td><td>admin</td><td>강형민</td><td>2023-12-12 14:32</td></tr><tr><td>3</td><td>TEST</td><td>테스트 권한</td><td>강형민</td><td>2025-02-17 01:22</td></tr><tr><td>4</td><td>VIEW</td><td>view</td><td>강형민</td><td>2025-02-24 11:27</td></tr><tr><td>5</td><td>VIEW</td><td>test</td><td>강형민</td><td>2025-02-24 11:27</td></tr></tbody></table> 권한 코드 USER 권한 명 작성자 작성 일자 페이지 1 / 1 10 시스템 관리자는 사용자 정보 및 사용자 권한, 상태를 수정 [1 - Auth Add] 입력한 내용을 초기화 [2 - Auth Delete] 선택한 권한 시스템에서 삭제 [3 - Auth Save] 입력한 내용을 데이터베이스에 수정 및 저장 [4 - Auth List] 시스템에서 권한 코드, 명, 작성자, 작성 일자 정보 제공 [5 - Auth Info] 권한 코드 : select box로 구성, USER, ADMIN, VIEW로 구성					권한코드	권한명	작성자	작성일자	1	USER	user	강형민	2023-12-12 14:31	2	ADMIN	admin	강형민	2023-12-12 14:32	3	TEST	테스트 권한	강형민	2025-02-17 01:22	4	VIEW	view	강형민	2025-02-24 11:27	5	VIEW	test	강형민	2025-02-24 11:27
	권한코드	권한명	작성자	작성일자																													
1	USER	user	강형민	2023-12-12 14:31																													
2	ADMIN	admin	강형민	2023-12-12 14:32																													
3	TEST	테스트 권한	강형민	2025-02-17 01:22																													
4	VIEW	view	강형민	2025-02-24 11:27																													
5	VIEW	test	강형민	2025-02-24 11:27																													

감사합니다

자세한 코드 : [Git](#)